

Organisation d'un projet frontend moderne

Objectifs de cette section

Cette section vous présente les bonnes pratiques d'organisation d'un projet frontend React/Next moderne. On y aborde les questions de structure, d'outillage, et de séparation des responsabilités dans un projet prêt pour la production. À la fin de cette section, vous serez capable de :

- Organiser vos fichiers selon les conventions actuelles.
- Distinguer les rôles des dossiers `components`, `pages`, `lib`, `styles`, etc.
- Utiliser un système de design cohérent (Atomic Design, tokens, etc.).
- Mettre en place un environnement de développement efficace.

1. La structure de base recommandée

Un projet bien structuré facilite la lecture, la collaboration et la montée en charge.

Exemple : structure Next.js + Atomic Design

```
/
├── components
│   ├── atoms
│   │   └── Button.jsx
│   ├── molecules
│   │   └── SearchBar.jsx
│   ├── organisms
│   │   └── ProductCard.jsx
│   └── templates
│       └── ProductPageTemplate.jsx
├── contexts
│   └── CartContext.jsx
├── hooks
│   └── useCart.js
├── lib
│   └── api.js
├── pages
│   ├── _app.jsx
│   ├── index.jsx
│   └── products
│       └── [id].jsx
├── public
│   └── logo.svg
├── styles
│   ├── globals.css
│   └── variables.css
├── .eslintrc.json
├── .prettierrc
├── package.json
└── README.md
```

2. Séparer les responsabilités

Chaque répertoire a une mission claire :

- `/pages` : point d'entrée des routes.
- `/components` : éléments visuels réutilisables.
- `/lib` ou `/services` : appels API, logique métier externe.
- `/styles` : CSS modules ou fichiers globaux.
- `/hooks` : custom hooks React.
- `/contexts` : providers et logique d'état global (React Context).

3. Outillage moderne

Un projet frontend moderne s'appuie sur une chaîne d'outils cohérente :

- **TypeScript** (optionnel mais recommandé)
- **ESLint + Prettier** pour la qualité du code
- **TailwindCSS** ou un système de design token pour la cohérence visuelle
- **React Query, Zustand** ou **Redux** pour la gestion de données
- **Jest** ou **Testing Library** pour les tests

Exemple : config ESLint/Prettier

`.eslintrc.json`

```
{
  "extends": [
    "next/core-web-vitals",
    "prettier"
  ],
  "plugins": [
    "react",
    "react-hooks"
  ],
  "rules": {
    "react/react-in-jsx-scope": "off",
    "react-hooks/rules-of-hooks": "error",
    "react-hooks/exhaustive-deps": "warn"
  }
}
```

`.prettierrc`

```
{
  "semi": true,
  "singleQuote": true,
  "printWidth": 100,
  "trailingComma": "all"
}
```

4. Bonnes pratiques générales

- Utiliser des noms de composants explicites (`ProductCard` plutôt que `Card`).
- Regrouper les composants avec leurs styles et tests si possible.
- Documenter l'interface des props (via commentaires, JSDoc, ou TypeScript).
- Penser en composants réutilisables dès le départ.
- Versionner le code avec Git, découper en branches logiques (`feature/`, `fix/`, etc.).

5. Exercice de mise en pratique

Consignes

1. Reprenez un projet Next.js existant ou démarrez un nouveau projet.
2. Réorganisez l'arborescence pour inclure :
 - `/components/atoms`, `/molecules`, etc.

- /lib ou /services
- /hooks, /contexts, /styles

3. Ajoutez un fichier de configuration ESLint + Prettier dans le projet.

Organisation attendue

[ARBORESCENCE : ORGANISATION FINALE PROJET]